

For the PX457, a `burgers.c` was provided with a simplified implementation of the 2D Burgers equation. A finite difference approximation was used with the equation

$$\frac{u_{n+1}^{i,j} - u_n^{i,j}}{\Delta t} = -u_n^{i,j} \left(\frac{u_n^{i+1,j} - u_n^{i-1,j}}{2\Delta x} + \frac{u_n^{i,j+1} - u_n^{i,j-1}}{2\Delta y} \right) + \nu \left(\frac{u_n^{i+1,j} + u_n^{i-1,j} - 2u_n^{i,j}}{\Delta x^2} + \frac{u_n^{i,j+1} + u_n^{i,j-1} - 2u_n^{i,j}}{\Delta y^2} \right) \quad (0.1)$$

The original code provided had several inefficiencies, with the primary task of the assignment being to optimise the program.

1. **Use of pow and precomputation:** the original code provided uses the `pow` function from the C standard library. This function is called at every grid point. Each function call is inefficient: the `pow` implementation uses $\text{pow}(x, y) = e^{y \log(x)}$, using both exponential and logarithmic operations with integers being converted to doubles for the calculation. Each `pow` call involves a division; all of these operations are slow, multi-cycle operations via microcode. To mitigate this, I precomputed the following constants:

```
const double inverse_dx = 1.0/dx;
const double inverse_dy = 1.0/dy;
const double half_inverse_dx = 0.5*inverse_dx;
const double half_inverse_dy = 0.5*inverse_dy;
const double inverse_dx_squared = inverse_dx * inverse_dx;
const double inverse_dy_squared = inverse_dy * inverse_dy;
```

This removed millions of unnecessary function calls and divisions per iteration, putting strength reduction into practice.

2. **Modulo operations instead of branching:** the original code used six `if/else` statements per grid point for handling bounds. This prevents the compiler from vectorising the loops, and can lead to pipeline stalling. By using modulo arithmetic, the computed indices wrap back round to match the dimensions of the grid, removing the need to use conditional branches.

```
/*-----*/
/* Loop over grid points */
/*-----*/
for(iy=0; iy<Ny; iy++) {
    // use modulo arithmetic to find next y index
    int next_y = (iy + 1) % Ny;
    int previous_y = (iy - 1 + Ny) % Ny;

    for(ix=0; ix<Nx; ix++) {
        // use modulo arithmetic to find next x index
        int next_x = (ix + 1) % Nx;
        int previous_x = (ix - 1 + Nx) % Nx;

        // get current and surrounding values
        double u_current = u[ix][iy];
        double u_right = u[next_x][iy];
        double u_left = u[previous_x][iy];
        double u_up = u[ix][next_y];
        double u_down = u[ix][previous_y];
```

```

        // compute derivatives
        // grad term (du/dx + du/dy)
        grad = (u_right - u_left) * half_inverse_dx + (u_up - u_down) * half_inverse_dy;
        laplacian = (u_left + u_right - 2.0 * u_current) * inverse_dx_squared
                    + (u_down + u_up - 2.0 * u_current) * inverse_dy_squared;

        // update u_new values
        u_new[ix][iy] = u_current - dt * u_current * grad + dt * nu * laplacian;
    }
}

```

3. **Pointer swapping:** the original code used a nested for loop to copy the variable `u_new` to `u`, with time complexity $O(n^2)$. By using an pointer swap, it changes memory ownership without the need to copy to data. This is done in $O(1)$ time.

```

// pointer swapping for O(1) allocation
double **temporary_grid = u;
u = u_new;
u_new = temporary_grid;

```

4. **Mistakes I made:** whilst modulo arithmetic is a simple topic, I accidentally got confused with defining my previous and next `x` and `y` variables. My `.dat` files differed in values significantly, but upon fixing this, my results matched the original `.dat` file.
5. **Reformatting the equation code:** as mentioned in the first point, division is cumbersome for the compiler in comparison to additions and multiplications. By changing the program so that it only uses additions and multiplications, it reduces the instruction count and reduces the execution time.
6. **Compiler optimisation:** the original `Makefile` used only `-Wall -O0`, with an average time of 9.3044 seconds for 10000 steps. By using `-O3 -march=native`, the average time for 10000 steps was 1.7899, a considerable improvement. The use of `-ffast-math` had a slightly higher average time per 10000 steps at 1.7963 seconds.